Meta-Learning: Comprehensive Lecture Notes

Reasoning, Hierarchical Models, and Hebbian TTA

Lectures 11–13 (February 12 – February 19)

IIIT Delhi

Compiled: February 27, 2026

Contents

1 Reasoning, Computation, and Hierarchical Models (February 12) 2

1.1 Reasoning and Computational Complexity . . . 2

1.1.1 Neural Architectures as Computational Models . . . 2

1.1.2 Chain-of-Thought as Iterated Computation . . . 3

1.2 The ARC Benchmark . . . 3

1.3 Hierarchical Reasoning Model (HRM) . . . 4

1.3.1 Formal Definition . . . 4

1.3.2 Intuition: Why Hierarchy Helps . . . 5

1.4 What Does Hierarchy and Recurrence Get? . . . 5

1.5 Aside: Cellular Automata as Neural Networks . . . 5

1.6 Deep Supervision for HRM Training . . . 6

1.6.1 HRM Pseudocode . . . 7

1.6.2 Training Loop . . . 7

1.7 Learning to Stop: Adaptive Computation . . . 8

1.7.1 Training the Halting Network . . . 8

1.8 Tiny Reasoning Model (TRM) . . . 9

2 MAML for ARC and Reasoning in LLMs (February 16) 9

2.1 MAML Applied to ARC . . . 9

2.1.1 Connecting HRM and TRM to MAML . . . 10

2.1.2 Primitives . . . 10

2.2 Reasoning in LLMs . . . 11

2.2.1 The LLM Training Pipeline 11
2.2.2 "Thinking" at Test Time 11
2.2.3 RLVR: Reinforcement Learning with Verifiable Rewards 12
2.2.4 RLVR in DeepSeek 12

3 Thinking LLMs, Agentic Reliability, and Hebbian TTA (February 19) 12
3.1 GRPO: Group Relative Policy Optimization 13
3.1.1 Connection to Meta-Learning 13
3.2 Agentic System Reliability 14
3.2.1 Reliability Analysis 14
3.3 Hebbian Learning for Test-Time Adaptation 15
3.3.1 Hebbian Learning Principles 15
3.3.2 Mathematical Formulation 15
3.3.3 Weight Norm Stability 16
3.3.4 Why Hebbian Learning for TTA? 16
3.4 Neuro-Modulated TTA via Hebbian Learning 17
3.4.1 Test-Time Behavior 17
3.4.2 Comparison with Other TTA Methods 17

1 Reasoning, Computation, and Hierarchical Models (February 12)

This lecture shifts focus from the meta-learning algorithms developed in Lectures 1–10 to the broader question of *reasoning* in neural networks. It establishes formal connections between computation theory and neural architectures, introduces hierarchical reasoning models for the ARC benchmark, and develops adaptive computation mechanisms.

1.1 Reasoning and Computational Complexity

The lecture begins with a fundamental question: **what computational problems can neural networks solve?** To answer this, it draws on the classical theory of computation.

Computation Hierarchy

- **Turing Machine (TM):** The most powerful model of computation. A TM consists of a Finite State Machine (FSM) with read/write access to an unbounded tape. It can compute any computable function.
- **One-Sided TM:** A TM where the tape head can only move in one direction (autoregressive). The FSM reads input sequentially and writes output sequentially. This is structurally equivalent to an autoregressive model.
- **Cost:** A standard TM can simulate a one-sided TM with at most polynomial overhead. That is, the two models are polynomially equivalent in computational power.

1.1.1 Neural Architectures as Computational Models

Different neural architectures correspond to different levels of the computational hierarchy:

Architecture	Computational Class	Language Class
RNN (finite precision)	Finite State Machine	Regular: $(ab)^*$
LSTM	Counter Machine (AC^0)	Context-free: $a^n b^n$
Transformer	Bounded-depth circuit	Depends on depth/width

RNNs as FSMs. With finite-precision arithmetic, an RNN's hidden state h_t can only take on finitely many distinct values. Each distinct hidden state corresponds to a state in a finite automaton. This means RNNs can recognize regular languages like $(ab)^*$ but *cannot* count – they cannot recognize $a^n b^n$ because that requires unbounded memory.

LSTMs as Counter Machines. The LSTM cell state C_t can act as a counter through its additive update rule $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. With sufficient precision, the LSTM can increment and decrement counters, enabling it to recognize context-free languages like $a^n b^n$. This places LSTMs in the AC^0 complexity class.

Transformers. The transformer processes all positions in parallel via self-attention. A single transformer layer computes a bounded-depth circuit over the input. The key constraint is that a

transformer with L layers and context length n processes tokens $x_0, \ldots, x_n, x_{n+1}, x_{n+2}, \ldots$ where the context window determines the effective "tape" length.

1.1.2 Chain-of-Thought as Iterated Computation

COT-Reasoning and Computational Power

Key insight: Autoregressive (AR) models with chain-of-thought (CoT) reasoning are *in principle* equivalent to Turing Machines.
A standard transformer processes input in a fixed number of forward passes (bounded depth). But with CoT, the model generates intermediate tokens that serve as a scratchpad. Each generated token feeds back as input, effectively creating **iteration in the input direction**.

$$\underbrace{x_0, \ldots, x_n}_{\text{input}} \rightarrow \underbrace{x_{n+1}, x_{n+2}, \ldots}_{\text{CoT tokens (scratchpad)}} \rightarrow y$$

This iteration transforms a bounded-depth computation into an *unbounded-depth* computation. The CoT tokens act as the tape of a Turing Machine, and the transformer's forward pass acts as the transition function.
Therefore: Transformer + CoT $\equiv$ TM (in principle).

The qualification "in principle" is important: in practice, transformers have finite context windows, finite precision, and finite training data. But the theoretical result shows that CoT is not merely a prompting trick – it fundamentally increases the computational class of problems that autoregressive models can solve.

1.2 The ARC Benchmark

The lecture introduces the Abstraction and Reasoning Corpus (ARC) as a concrete testbed for reasoning capabilities.

ARC-1 (Abstraction and Reasoning Corpus)

ARC tasks consist of input-output grid pairs. Each task has:

- **Training examples:** 2–5 pairs of (input grid, output grid)
- **Test example:** An input grid; the model must produce the correct output grid

The grids are 2D arrays of colored cells (typically $\leq 30 \times 30$). The transformation rule must be *inferred* from the few training examples and applied to the test input. This is fundamentally a few-shot learning problem.

Benchmark results at the time of the lecture:

Model	Accuracy
GPT-4x	15%
o1	20%
o3	85%
HRM	40%
Tested on private eval	32%

ARC is significant for meta-learning because each task is a standalone learning problem: the model must extract a rule from a handful of examples and generalize it. This is exactly the meta-learning setup – the task distribution is over visual transformation rules.

1.3 Hierarchical Reasoning Model (HRM)

The lecture introduces the Hierarchical Reasoning Model (HRM) as an architecture designed for iterative, multi-scale reasoning on tasks like ARC.

HRM Architecture

The HRM has three key design principles:

1. **Recurrent connections:** The model iterates over its own output, refining predictions over multiple steps.
2. **Multiple time-scales:** A *low-level* (fast) module f_L updates every step, while a *high-level* (slow) module f_H updates only periodically (every T steps).
3. **Feedback connections:** The high-level state feeds back into the low-level computation, enabling top-down guidance.

1.3.1 Formal Definition

Let $\tilde{x} = f_I(x, \theta)$ be the initial encoding of input x. The HRM maintains two state variables:

- $Z_L^i \in \mathbb{R}^{d_L}$ – the **low-level state** (fast timescale), updated at every iteration i
- $Z_H^i \in \mathbb{R}^{d_H}$ – the **high-level state** (slow timescale), updated every T iterations

The update rules are:

$$Z_L^i \leftarrow f_L(Z_L^{i-1},\, Z_H^{i-1},\, \tilde{x},\, \theta) \tag{1}$$

$$Z_H^i \leftarrow \begin{cases} f_H(Z_L^{i-1},\, Z_H^{i-1},\, \theta) & \text{if } i \bmod T = 0 \\ Z_H^{i-1} & \text{otherwise} \end{cases} \tag{2}$$

After NT total iterations (where N is the number of high-level updates), the output is:

$$y \leftarrow f_O(Z_H^{NT}, \theta) \tag{3}$$

1.3.2 Intuition: Why Hierarchy Helps

The two timescales serve different purposes:

Low-level module f_L: Performs rapid, local refinement. It processes the input encoding $\tilde{x}$ at each step, making fine-grained adjustments to its state. This is analogous to "thinking fast" – quick pattern matching and local computation.

High-level module f_H: Performs slower, abstract reasoning. It integrates information accumulated by f_L over T steps and updates a compressed, high-level summary. This is analogous to "thinking slow" – deliberate, abstract reasoning that guides the low-level processing.

The feedback from Z_H to f_L (Equation 1) is crucial: the high-level module can *steer* the low-level computation, directing attention to relevant features or correcting errors identified at the abstract level.

1.4 What Does Hierarchy and Recurrence Get?

The lecture contrasts the behavior of recurrent (RNN-like) and feedforward (DNN) architectures:

Residual Behavior in RNNs vs. DNNs

RNN (recurrence over steps):

- The residual (prediction error) typically *decreases* with more recurrent steps.
- Each iteration refines the prediction, reducing the gap between the current output and the target.
- The residual curve decreases monotonically (in well-trained models).

DNN (depth over layers):

- The residual may *increase* with depth if the network is not well-regularized.
- Deeper layers can overfit or lose information, leading to degraded representations.

When hierarchy *and* recurrence are combined (as in HRM), we get a richer picture. The residual or gradient signal shows oscillating behavior at two frequencies: the fast oscillations correspond to the low-level updates (every step), while the slow envelope corresponds to the high-level updates (every T steps). Over layers (in the high-level direction), the residual decreases as the abstract representation improves.

1.5 Aside: Cellular Automata as Neural Networks

The lecture draws an aside connecting recurrent neural networks to cellular automata (CA).

Cellular Automata NN

A cellular automaton operates on a grid where each cell $x_{i,j}^t$ is updated according to a local rule f_θ applied to its neighborhood:

$$x_{i,j}^t = f_\theta(x_{i,j}^{t-1}) \tag{4}$$

This is *structurally identical* to a convolutional RNN: the same local function is applied at every spatial position (weight sharing) and iterated over time steps.
Example: Conway's Game of Life is a cellular automaton with a fixed rule. A neural CA replaces the fixed rule with a learned function f_θ.

Two key properties of iterated CA dynamics are relevant:

(A) Convergence to fixed points. Under certain conditions, the iterated application $\tilde{x}^0, \tilde{x}^1, \ldots$ converges to a fixed point x^* satisfying:

$$x^* = f_\theta(x^*) \tag{5}$$

This is significant because it means the model can automatically determine how much computation is needed – iterate until convergence.

(B) Stability after convergence. Once the system reaches x^*, further iterations leave it unchanged: $x^{t+N} = x^*$ for all $N > 0$. The system "halts" naturally.

The connection to ARC is direct: ARC tasks involve spatial grids transformed by local rules – exactly the setting of cellular automata. The HRM can be viewed as a hierarchical, learned cellular automaton applied to ARC grids.

1.6 Deep Supervision for HRM Training

Training a model with many recurrent steps is challenging due to vanishing/exploding gradients through the long chain of iterations. The lecture introduces **deep supervision** as the training strategy for HRM.

Deep Supervision

Instead of computing loss only at the final iteration, compute loss at the end of each *segment* of NT steps.
Let m index segments. After segment m (i.e., after $m \cdot NT$ total iterations):

$$Z^m = (Z_L^{m \cdot NT},\, Z_H^{m \cdot NT}) \tag{6}$$
$$Z^m \leftarrow \mathrm{HRM}(Z^{m-1}, x, \theta) \tag{7}$$
$$y^m = f_O(Z^m, \theta) \tag{8}$$
$$\mathcal{L}^m = \mathcal{L}(y^m, y_{\text{true}}) \tag{9}$$

The parameter update uses:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta \mathcal{L}^m \tag{10}$$

Key trick: detach between segments. After each segment, Z^m is detached from the computation graph (i.e., Z.detach() in PyTorch). This means gradients do not flow backward

through the entire chain of segments. Each segment is trained independently, assuming the previous state is a constant.

Why detach? If we backpropagated through all M segments, we would need to store the computation graph for $M \times NT$ steps – potentially thousands of iterations. This is both memory-prohibitive and numerically unstable. By detaching, we:

1. Limit the backpropagation depth to a single segment (NT steps)
2. Avoid vanishing/exploding gradients across segments
3. Still provide supervision at every segment, ensuring the model learns to produce good outputs at intermediate stages

This is analogous to truncated backpropagation through time (TBPTT) in RNN training. The assumption is that if the model converges within each segment, then $\nabla_\theta \mathcal{L} \approx \nabla_\theta Z^*$, where Z^* is the fixed point – and differentiating through the fixed point equation does not require the full unrolled computation.

1.6.1 HRM Pseudocode

HRM Forward Pass: Pseudocode

```
function HRM(Z, x̃)
    Z_H, Z_L ← Z
    with no_grad:
    for i = 0 to NT − 2 do
        Z_L ← f_L(Z_L, Z_H, x̃)
        if (i + 1) mod T = 0 then
            Z_H ← f_H(Z_L, Z_H)
        end if
    end for
                                        ▷ Final step with gradients enabled:
    Z_L ← f_L(Z_H, Z_L, x̃)
    Z_H ← f_H(Z_H, Z_L)
    y ← f_O(Z_H)
    return Z, y
end function
```

1.6.2 Training Loop

HRM Training with Deep Supervision

```
Given: Many (x, y) training pairs
while not converged do
    for m = 1 to M steps of supervision do
        Z, y ← HRM(Z, x̃)                ▷ One segment forward pass
```

```
        L ← CE(y, y_true)
        Z.detach()                          ▷ Stop gradient flow between segments
        optimizer.step()
        optimizer.zero_grad()
    end for
end while
```

The bottom portion of the slide shows that the architecture processes the input through multiple segments, each producing a loss. The segments progressively increase in temporal depth – early segments use fewer low-level/high-level iterations, and the final segments capture the full iterative computation.

1.7 Learning to Stop: Adaptive Computation

A critical question for iterative models: **how many iterations are enough?** Different inputs may require different amounts of computation. The lecture introduces a learned halting mechanism.

Learning to Stop

At each segment m, the model produces:

- A prediction $y^m = f_O(Z^m, \theta)$
- A halting probability $q^m = Q(Z^m, \phi)$, where Q is a learned halting network with parameters ϕ

Halting rule:

$$\text{If } q^m > 0.5, \text{ halt and output } y^m. \text{ Otherwise, continue to segment } m+1. \tag{11}$$

The maximum number of segments is capped at $M_{\max}$ to ensure termination.

1.7.1 Training the Halting Network

The halting network Q must learn *when* the model's prediction is correct. The training signal is constructed as follows:

For each segment m, define the target r^m:

$$r^m = \begin{cases} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = [p_{\text{stop}},\, p_{\text{cont}}] & \text{if } y^m = y_{\text{true}} \quad \text{(prediction is correct} \rightarrow \text{should stop)} \\ \begin{bmatrix} q^m \\ \sigma(q^m) \end{bmatrix} & \text{if } y^m \neq y_{\text{true}} \quad \text{(prediction is wrong} \rightarrow \text{should continue)} \end{cases} \tag{12}$$

The total loss combines prediction and halting:

$$\mathcal{L}_{\text{total}} = \mathcal{L}(y^m, y_{\text{true}}) + \text{BCE}(q^m, r^m) \tag{13}$$

where BCE is binary cross-entropy. The halting network is trained to predict whether the current prediction matches the ground truth. At test time, the halting network decides when to stop without access to y_{true}.

This is a form of **learned adaptive computation**: easy inputs halt after few segments (less computation), while hard inputs iterate for more segments (more computation). The connection to meta-learning is that the halting mechanism is itself meta-learned – trained across many examples to decide when adaptation is sufficient.

1.8 Tiny Reasoning Model (TRM)

The lecture presents a simplified variant of HRM called the Tiny Reasoning Model (TRM), designed to be more practical while retaining the key ideas.

TRM: Tiny Reasoning Model

The TRM makes three simplifications relative to HRM:

1. **Do not assume convergence.** Instead of relying on fixed-point convergence, use full backpropagation through time (BPTT) for all T steps. This avoids the need for the detach trick but limits the total number of iterations.

2. **Z_H is essentially y.** The high-level state directly represents the output prediction, eliminating the separate output head f_O. This simplifies the architecture by merging the high-level state and the output.

3. **Simpler Q (halting).** The halting function operates directly on y: $Q(y) \to \{0, 1\}$. Stop if $Q(y) > 0.5$. The halting loss uses BCE with the correctness signal: $\text{BCE}(y = y_{\text{true}})$.

Architecture: The function f is implemented as a *transformer decoder only*. The grid is serialized into a 1D sequence.
On ARC: HRM achieves 40% on the public eval; TRM with these simplifications achieves 32% on the private eval.

The key insight is that for ARC, the grid-to-grid transformation can be cast as a sequence-to-sequence problem (by flattening the 2D grid to 1D), and a decoder-only transformer can serve as the iterative reasoning module. The "task ID" – which transformation rule to apply – is implicitly encoded in the training examples (the support set) provided as context.

2 MAML for ARC and Reasoning in LLMs (February 16)

This lecture connects the meta-learning framework (specifically MAML) to the ARC benchmark and then pivots to discuss how reasoning emerges in large language models through reinforcement learning.

2.1 MAML Applied to ARC

The lecture revisits MAML and shows how it naturally applies to ARC tasks.

MAML for ARC

Each ARC task T has the standard meta-learning structure:

- $D_{\text{train}} = \{(\text{input grid}_i, \text{output grid}_i)\}_{i=1}^{K}$ – the demonstration pairs (support set)
- $D_{\text{test}} = \{(\text{input grid}_{\text{test}}, ?)\}$ – the test input (query set)

Inner loop: Given parameters θ, adapt on the training examples:

$$y' = f_\theta(X_{\text{train}}) \tag{14}$$
$$\mathcal{L}^I = \mathcal{L}(y', Y_{\text{train}}) \tag{15}$$
$$\theta' \leftarrow \theta - \epsilon \nabla_\theta \mathcal{L}^I \tag{16}$$

Outer loop: Evaluate the adapted model on the test example:

$$y = f_{\theta'}(X_{\text{test}}) \tag{17}$$
$$\mathcal{L}^O = \mathcal{L}(y, Y_{\text{test}}) \tag{18}$$

The meta-update optimizes θ so that the *adapted* parameters θ' produce correct test outputs.

2.1.1 Connecting HRM and TRM to MAML

The HRM and TRM can both be placed within the MAML framework:

- **HRM as inner loop:** The iterative refinement of Z_L, Z_H over NT steps serves as the "adaptation" phase. The halting function $Q(y, x)$ determines when adaptation is complete. MAML provides the meta-learning signal for training θ.
- **TRM as inner loop:** Similarly, the TRM's iterative transformer computation serves as the inner loop, with the simplified halting mechanism.

The ARC dataset naturally splits into a training set and evaluation set. The HRM uses only the training examples (D_{train} in MAML terms) for its iterative computation, and is evaluated on the test example (D_{test}).

2.1.2 Primitives

The lecture briefly discusses **primitives** – basic visual operations (translation, rotation, color change, flood fill, etc.) that compose to form ARC task solutions. The idea is that instead of learning transformation rules from scratch, the model could learn a *library of primitives* and learn to compose them for each task. This achieved 32% on the benchmark, suggesting that while compositional approaches have merit, learning the right primitives and composition rules remains challenging.

2.2 Reasoning in LLMs

The lecture transitions to discuss how reasoning capabilities emerge in large language models, covering the standard LLM training pipeline and then focusing on reinforcement learning approaches.

2.2.1 The LLM Training Pipeline

A modern LLM is trained in three phases:

LLM Training Phases

1. **SFT (Supervised Fine-Tuning) / Pre-training:** Train the base model $p_\theta(x \mid q)$ on large text corpora. The model learns language patterns, factual knowledge, and basic reasoning patterns.
2. **Instruction Tuning (mid-training):** Fine-tune on instruction-following data to align the model's behavior with user expectations. The model learns to follow prompts and produce helpful responses.
3. **RLHF (Reinforcement Learning from Human Feedback) / Post-training:** Further refine the model using human preference data. Multiple outputs are generated, ranked by humans, and used to train a reward model. The policy is then optimized to maximize the learned reward.

2.2.2 "Thinking" at Test Time

The lecture introduces the concept of test-time reasoning ("thinking") in LLMs, drawing a direct parallel to the meta-learning framework.

Chain-of-Thought as Test-Time Adaptation

(A) Chain-of-Thought (CoT): Given a query q, the model generates a sequence of reasoning tokens before producing the final answer:

$$q \to \underbrace{\text{thought}_1, \text{thought}_2, \ldots, \text{thought}_n}_{\text{chain of thought}} \to \text{answer}$$

During CoT, the model generates intermediate hypotheses and refines them. This is structurally identical to the iterative computation in HRM – the "thinking tokens" serve as the recurrent state being refined.

Verifiable outputs: A key property that enables RL-based training of reasoning is *verifiability*. For some tasks (math, code, formal logic), we can define a verifier function:

$$f(y) \in \{\text{True}, \text{False}\}$$

that checks whether the output y is correct. This provides a ground-truth reward signal without human annotation.

The connection to the meta-learning framework is deep:

- CoT is **similar to TTA** (Test-Time Adaptation): the model adapts its computation at test time based on the specific input.
- If we train the model to perform this adaptation well across many tasks, we are **doing meta-learning**: learning to adapt at test time.

2.2.3 RLVR: Reinforcement Learning with Verifiable Rewards

The lecture introduces RLVR as the training paradigm for reasoning LLMs.

RLVR (Reinforcement Learning with Verifiable Rewards)

For tasks with verifiable answers (math, code, logic):

1. Given a query q, generate G candidate outputs $o_1, \ldots, o_G$ from $\pi_\theta(o \mid q)$
2. Score each output: $r_i = f(o_i) \in \{0, 1\}$ (binary correctness)
3. Use RL to maximize expected reward: $\max_\theta \frac{1}{G} \sum_{i=1}^{G} \pi_\theta(o_i \mid q) \cdot r_i$

This is *SAME as HRM but in the RL setting*: the model iterates (via CoT) to produce an answer, receives a reward, and updates to improve future reasoning.

2.2.4 RLVR in DeepSeek

The lecture specifically discusses the DeepSeek approach to RLVR:

DeepSeek RLVR

Given query q, sample G outputs with rewards:

$$q \to \pi_\theta(o \mid q) \to o_1(r_1),\ o_2(r_2),\ \ldots,\ o_G(r_G)$$

The policy ratio is:

$$\rho_i = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}$$

This ratio measures how much the current policy has changed relative to the policy that generated the samples. It is used in the PPO/GRPO clipped objective (detailed in the next lecture).

3 Thinking LLMs, Agentic Reliability, and Hebbian TTA (February 19)

This lecture completes the reasoning-in-LLMs arc with the GRPO algorithm, develops a formal framework for agentic system reliability, and introduces Hebbian learning as a biologically-inspired mechanism for test-time adaptation.

3.1 GRPO: Group Relative Policy Optimization

The lecture formalizes the RL training objective for "thinking" LLMs.

GRPO (Group Relative Policy Optimization)

Given query q, sample G candidate outputs from $\pi_{\theta_{\text{old}}}$:

$$o_1, o_2, \ldots, o_G \sim \pi_{\theta_{\text{old}}}(o \mid q), \quad \text{with rewards } r_1, \ldots, r_G$$

Normal RL objective:

$$\max_{\theta} \frac{1}{G} \sum_{i=1}^{G} \pi_\theta(o_i \mid q) \cdot r_i$$

Advantage computation (group relative):

$$A_i = \frac{r_i - \text{avg}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

The advantage A_i measures how much better output o_i is compared to the *group average*. This is the "group relative" part – the baseline is computed from the group of samples for the same query, not from a separate value network.
GRPO clipped objective:

$$J = \frac{1}{G} \sum_{i=1}^{G} \min\left[\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)} \cdot A_i, \;\; \text{clip}\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}, 1-\epsilon, 1+\epsilon \right) \cdot A_i \right] \tag{19}$$

Maximize this objective: $\theta \leftarrow \theta + \epsilon \nabla_\theta J$.
The clipping prevents the policy from changing too much in a single update (as in PPO), ensuring training stability.

3.1.1 Connection to Meta-Learning

The lecture draws explicit connections between GRPO/RLVR and the meta-learning framework:

RLVR ↔ Meta-Learning

- **Inference strategies:**
  - **Best-of-N:** Generate N candidate answers, use the verifier to select the best. This is a form of test-time compute scaling without weight updates.
  - **TTA (Test-Time Adaptation):** Similar to training data – adapt the model at test time using the reward signal. This is TTA applied to reasoning.
- **The training pipeline:**
  - SL (Supervised Learning) → IT (Instruction Tuning) → PT (Post-Training via RLVR)

- The post-training phase *is* meta-learning: training the model to adapt (think/reason) effectively at test time.

3.2 Agentic System Reliability

The lecture introduces a formal framework for analyzing the reliability of LLM-based agentic systems that use verifiers.

Agentic System with Verifier

Setup:

- An LLM generates an output o for input query q
- A verifier V evaluates the output: $V(o) \in \{\text{Yes}, \text{No}\}$
- $p(\text{verifier is correct}) = p$ – the probability that V correctly judges whether o is acceptable
- $p(V \text{ is incorrect}) = q = 1 - p$

Sequential verification: Run the verifier multiple times. After each run, tally the "Yes" and "No" votes. Continue until:

$$|\#\text{yes} - \#\text{no}| \geq K$$

where K is the confidence threshold.

3.2.1 Reliability Analysis

This is a form of **Sequential Probability Ratio Test (SPRT)** or random walk with absorbing barriers. The key results are:

If the process terminates (which it does with probability 1 when $p \neq 0.5$):

$$p(\text{correct decision}) = \frac{1}{1 + \left(\frac{1-p}{p}\right)^K} \tag{20}$$

This formula shows that:

- When $p > 0.5$ (verifier is better than random) and K is large, the probability of a correct decision approaches 1 exponentially fast.
- Even a weak verifier (p slightly above 0.5) can be amplified to arbitrarily high reliability by increasing K.

Expected length of the procedure:

$$\mathbb{E}[\text{length}] \approx \frac{K}{2(p-q)} = \frac{K}{2(2p-1)} \sim K \tag{21}$$

The expected number of verification rounds scales linearly with K. When the verifier is strong (p close to 1), the denominator $2(2p-1)$ is close to 2, so $\mathbb{E}[\text{length}] \approx K/2$. When the verifier is weak (p close to 0.5), the expected length grows.

This framework is practically important: it shows that **reliable agentic systems can be built from unreliable components** by using repeated verification with majority voting. The cost is linear in the confidence parameter K.

3.3 Hebbian Learning for Test-Time Adaptation

The lecture introduces Hebbian learning as a biologically-inspired, unsupervised adaptation mechanism for TTA. This connects the neuroscience of learning to the meta-learning framework.

3.3.1 Hebbian Learning Principles

Hebbian Learning

Two foundational principles:

1. **"Neurons that fire together wire together" (Hebb's Rule):** If a presynaptic neuron and a postsynaptic neuron are both active, the connection (weight) between them is strengthened.
2. **Lateral inhibition:** Active neurons suppress nearby inactive neurons, creating competition and sparsity.

3.3.2 Mathematical Formulation

Consider a single neuron with:

- Input: x_i (presynaptic activities)
- Weights: w_i
- Output: $u = w \cdot x = \sum_i w_i x_i$ (membrane potential)
- Activation: $y = \text{Softmax}(\Sigma w_i x_i)$ (competitive, via lateral inhibition)

The weight update proportional to the Hebbian rule:

$$\Delta w_i \propto u \cdot x_i \tag{22}$$

However, naive Hebbian learning suffers from unbounded weight growth. The **Oja rule** (a normalized variant from Hopfield networks) resolves this:

Oja's Rule / Hopfield-Style Update

$$\Delta w_i = \epsilon\, y_i(x_i - w_i u) \tag{23}$$

or equivalently in vector form:

$$\frac{dw}{dt} = \epsilon\, y(x - wu) \quad \in \mathbb{R}^n \tag{24}$$

The term $(x_i - w_i u)$ acts as a self-normalizing mechanism. The $w_i u$ subtraction prevents runaway weight growth by penalizing weights proportional to their contribution to the output.

3.3.3 Weight Norm Stability

The lecture proves that Oja's rule automatically normalizes the weight vector. Computing the time derivative of $\|w\|^2$:

$$\frac{d\|w\|^2}{dt} = 2w \cdot \frac{dw}{dt} = 2w \cdot \epsilon\, y(x - wu) \tag{25}$$

$$= 2\epsilon\, y\, u\, (1 - w^\top w) \tag{26}$$

$$= 2\epsilon\, y\, u\, (1 - \|w\|^2) \tag{27}$$

This shows that:

- If $\|w\|^2 < 1$: the derivative is positive (assuming $y, u > 0$), so $\|w\|$ increases.
- If $\|w\|^2 > 1$: the derivative is negative, so $\|w\|$ decreases.
- If $\|w\|^2 = 1$: the derivative is zero – **equilibrium**.

The weight norm converges to 1 automatically, regardless of initialization. This self-stabilization is a key advantage of Hebbian learning for TTA: no learning rate schedule or explicit normalization is needed.

3.3.4 Why Hebbian Learning for TTA?

The connection to TTA is compelling:

1. **No labels required:** Hebbian learning is entirely unsupervised – it depends only on the activations x and y, not on any ground-truth labels. This makes it directly applicable to TTA where labels are unavailable.
2. **Local computation:** Each weight update depends only on the pre- and post-synaptic activities, not on global error signals. This is computationally efficient and biologically plausible.
3. **Automatic normalization:** The self-stabilizing property of Oja's rule means the adaptation is inherently stable – no risk of divergent weight growth during test-time adaptation.

3.4 Neuro-Modulated TTA via Hebbian Learning

The lecture culminates with a complete TTA architecture that combines Hebbian learning with neural modulation.

Neuro-Modulated TTA Architecture

The model consists of three sequential modules:

$$x \to \boxed{H} \to \boxed{N} \to \boxed{C} \to y$$

- $\boxed{H}$ – **Hebbian layer:** Updated at test time via Hebbian learning (Oja's rule). Does *not* use labels y or y_{true}.
- $\boxed{N}$ – **Neuro-modulation layer:** A standard neural network layer that modulates the Hebbian layer's output. Trained during the training phase but *frozen* at test time.
- $\boxed{C}$ – **Classifier:** Maps the modulated representation to class predictions. Frozen at test time.

3.4.1 Test-Time Behavior

At test time, *only the Hebbian layer H is updated.* The key properties:

- The Hebbian layer adapts its weights to the test distribution using only the input activations – no labels or pseudo-labels needed.
- The neuro-modulation layer N transforms the Hebbian representations into a form useful for classification. It was trained to work with Hebbian-adapted features during training.
- The classifier C produces predictions from the modulated features.

The loss used to *evaluate* (not to update H) is the entropy of the prediction:

$$\mathcal{L} \propto H(y) = -\sum_{k=1}^{C} y_k \log y_k \tag{28}$$

But critically, **the Hebbian layer does not use this loss for its updates**. It uses only the local Hebbian rule (Equation 23). The entropy is used for evaluation and for training the neuro-modulation layer during the training phase.

3.4.2 Comparison with Other TTA Methods

Method	TTA Signal	What is Updated
TENT	Entropy minimization only	Batch normalization parameters
TTT (Test-Time Training)	Reconstruction loss	Self-supervised head + shared encoder
Neuro-Modulated Hebbian	Hebbian rule (no loss)	Hebbian layer weights

TENT (Wang et al., 2021) adapts only the batch normalization parameters by minimizing prediction entropy. It is simple but limited – entropy minimization can reinforce incorrect predictions.

TTT (Sun et al., 2020) uses a reconstruction loss (self-supervised) to adapt the shared encoder at test time. This is the dual-head architecture from Lecture 8.

Neuro-Modulated Hebbian TTA is distinct in that it does not use *any* loss function for adaptation. The Hebbian rule is a fixed update rule that automatically adjusts to the input distribution. The neuro-modulation layer provides the "meta-learned" component – trained to make the Hebbian adaptation effective for downstream classification.

The lecture mentions evaluation on CIFAR-100 with various corruptions (ImageNet-31, test data corruptions), where the Hebbian TTA approach shows competitive performance with TENT and TTT while being simpler and more stable.

Summary: Lectures 11–13

Summary

Lecture 11 (Feb 12): Computational complexity of neural architectures (FSM → counter machine → TM). CoT as iterated computation – Transformer + CoT ≡ TM in principle. The ARC benchmark as a meta-learning problem. Hierarchical Reasoning Model (HRM) with dual timescales. Deep supervision with detached gradients. Learning to stop via adaptive halting. TRM as a simplified variant.
Lecture 12 (Feb 16): MAML applied to ARC tasks. Compositional primitives. The LLM training pipeline (SFT → IT → RLHF). Reasoning as test-time computation. RLVR for training thinking models. DeepSeek's approach with policy ratios.
Lecture 13 (Feb 19): GRPO algorithm – group-relative advantage with clipped objectives. Agentic system reliability via sequential verification (SPRT). Hebbian learning: Oja's rule with automatic weight normalization. Neuro-modulated TTA architecture. Comparison with TENT and TTT.

These three lectures extend the meta-learning framework in two major directions:

1. Reasoning as iterated meta-learning. The HRM/TRM architectures show that reasoning on tasks like ARC is fundamentally an iterative adaptation process – the model refines its answer over multiple steps, guided by hierarchical structure and learned halting. Chain-of-thought in LLMs is the same process in the language domain. RLVR/GRPO provide the training framework. The deepest connection: *test-time reasoning is test-time adaptation is meta-learning.*

2. Biologically-inspired TTA. Hebbian learning provides a fundamentally different approach to test-time adaptation: instead of gradient-based updates using engineered loss functions (entropy, reconstruction), the Hebbian rule adapts weights using only local activity correlations. The self-normalizing property of Oja's rule ensures stability without learning rate tuning. When combined with a meta-learned neuro-modulation layer, this creates a TTA system where the *adaptation mechanism* is fixed (Hebbian) but the *context for adaptation* is learned.

The Unifying Thread

Across all three lectures, the central theme is: **how to allocate computation adaptively at test time.**

- HRM/TRM: Iterate until a learned halting function says "stop"
- CoT in LLMs: Generate reasoning tokens until the answer crystallizes
- GRPO/RLVR: Train the model so that its test-time computation (thinking) is effective
- Agentic verification: Repeat verification until confidence is sufficient
- Hebbian TTA: Adapt representations to the test distribution via unsupervised, self-stabilizing updates

This is meta-learning at its most general: **learning how much and what kind of computation to perform for each new problem.**